EduPredict — Student Performance Prediction & Analysis

Stack & Scale Remote Internship 2026 | AI Engineering — Week 02 Comprehensive Project Documentation

Intern Name	Alishan
Project Domain	AI Engineering & Applied Machine Learning (Week 02 Guided Intermediate)
Submission Delivery	Friday, 6 November 2026
GitHub Repository	https://github.com/Alishan45/EduPredict

1. Weekly Accomplishment Summary (10 Key Points)

During Week 02 of the Stack & Scale AI Engineering internship, the EduPredict machine learning system was conceived, implemented, evaluated, and documented following professional ML engineering standards. The following 10 points summarize the completed work:

- **1. Dataset Ingestion & Archive Engineering:** Acquired and verified the UCI Student Performance benchmark dataset across both cohorts—Mathematics (395 records) and Portuguese Language (649 records). Resolved a nested ZIP extraction bug in the data pipeline to guarantee seamless, zero-manual-intervention reproducibility.
- **2. Pedagogical Target Formulation:** Constructed a 3-tier discrete academic performance target based on Portugal's official secondary grading scale (0–20): *'Needs Support'* (0–9), *'Satisfactory'* (10–13), and *'Strong'* (14–20). This avoids arbitrary thresholding while reflecting actionable educational intervention tiers.
- **3. Data Hygiene & Quality Assurance:** Conducted thorough data audits verifying 0 missing values, 0 duplicate rows, and correct type encodings across all 33 demographic, familial, and behavioral attributes.
- **4. Strict Zero-Leakage Architecture:** Enforced target leakage safeguards by completely dropping final grade G3 (the label source) and intermediate trimester exams G1 and G2 from pre-term feature matrices, preventing artificial grade interpolation.
- **5. Leak-Free ColumnTransformer Pipeline:** Encapsulated preprocessing inside a scikit-learn Pipeline. Handled numerical attributes with median imputation and standardization, and categorical attributes with most-frequent imputation and One-Hot Encoding, strictly fitted on training splits only.
- **6. Majority Benchmark Establishment:** Constructed a DummyClassifier benchmark predicting the majority class (*'Satisfactory'*, 41.8% baseline) to ensure all machine learning models are evaluated against realistic statistical reference points.

- **7. Multi-Model Architecture & Tuning:** Engineered both a constrained DecisionTreeClassifier (depth=3, min_samples_leaf=5) and a regularized LogisticRegression classifier (C=0.1, L2 penalty), analyzing their bias-variance trade-offs.
- **8. 5-Fold Stratified Cross-Validation:** Conducted 5-fold stratified cross-validation on training data to establish out-of-fold generalization bounds (Decision Tree: 50.9% ± 3.9%, Logistic Regression: 47.4% ± 4.5%).
- **9. Correlation Heatmap & Feature Selection:** Generated Pearson correlation heatmaps that uncovered the strongest clean predictors (past failures r=-0.36, mother's education r=+0.22, higher education ambition r=+0.18). Pruning noisy attributes improved test accuracy to 48.1% (+6.3% over baseline).
- **10. Multi-Tab Streamlit App & Dual-Timeline Deployment:** Built an interactive Streamlit application featuring both Pre-Term Early Warning Mode (~48% Acc, survey only) and Mid-Term Tracking Mode (~89% Acc, G1/G2 included), complete with profile presets and responsible AI guidelines.

2. Code Implementation Screenshots

The codebase is structured into modular components: dataset handling (`src/data.py`), model pipelining (`src/model.py`), automated training CLI (`train_model.py`), and web simulation dashboard (`app.py`).

2.1 Data Ingestion & Extraction Pipeline (src/data.py)

Handles automatic dataset downloading from the UCI repository, unpacks nested ZIP archives, and structures CSV data locally with project-root anchoring.

```python
from pathlib import Path
import io, zipfile, requests, pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = PROJECT_ROOT / "data"

def load_student_data(subject="mat", data_dir=None):
    """Download and load UCI dataset with robust nested-zip extraction."""
    target_dir = Path(data_dir) if data_dir else DATA_DIR
    csv_path = target_dir / f"student-{subject}.csv"
    if not csv_path.exists():
        archive = target_dir / "student-performance.zip"
        if not archive.exists():
            resp = requests.get(UCI_URL, timeout=30)
            archive.write_bytes(resp.content)
        _extract_archive(archive, target_dir)
    return pd.read_csv(csv_path, sep=";")
```

Figure 2.1: src/data.py — Automated UCI Dataset Retrieval and Nested ZIP Extractor

2.2 Preprocessing & Pipeline Construction (src/model.py)

Constructs leak-free pipelines using ColumnTransformers for median imputation, scaling, and one-hot encoding fitted strictly on training data.

```python
def build_pipeline(X: pd.DataFrame, model_type: str = "decision_tree", **kwargs):
    """Construct leak-free Pipeline: imputation, scaling, OHE, and classifier."""
    numeric_features = X.select_dtypes(include=["number"]).columns.tolist()
    categorical_features = X.select_dtypes(exclude=["number"]).columns.tolist()

    numeric_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])
    categorical_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ])
    preprocessing = ColumnTransformer([
        ("numeric", numeric_pipe, numeric_features),
        ("categorical", categorical_pipe, categorical_features)
    ])
    if model_type == "decision_tree":
        clf = DecisionTreeClassifier(max_depth=3, min_samples_leaf=5, random_state=42)
    elif model_type == "logistic_regression":
        clf = LogisticRegression(C=0.1, max_iter=1000, random_state=42)
    return Pipeline([("preprocessing", preprocessing), ("classifier", clf)])
```

Figure 2.2: src/model.py — Preprocessing ColumnTransformer and Pipeline Construction

2.3 CLI Training, Validation & Serialization (train_model.py)

Command-line script executing stratified splits, baseline comparisons, cross-validation, confusion matrices, and joblib model persistence.

```python
def main():
    parser = argparse.ArgumentParser(description="EduPredict Training & Evaluation")
    parser.add_argument("--subject", choices=["mat", "por"], default="mat")
    parser.add_argument("--model", choices=["decision_tree", "logistic_regression"], default="decision_tree")
    parser.add_argument("--include-grades", action="store_true", help="Include G1 & G2 for ~89% acc")
    args = parser.parse_args()

    df = load_student_data(args.subject)
    results = train_and_save(
        df=df, output_path=args.output, model_type=args.model,
        include_prior_grades=args.include_grades
    )
    print(f"Model accuracy: {results['accuracy']:.3f}")
    print(f"Majority baseline: {results['baseline_accuracy']:.3f}")
    print(results["report"])
```

Figure 2.3: train_model.py — Command-Line Interface for Training, Cross-Validation & Serialization

2.4 Interactive Web Application (app.py)

Streamlit web application implementing sidebar timeline controllers, preset student profile loaders, side-by-side probability estimates, and responsible AI guardrails.

```python
# Streamlit Deployment Mode Controller
prediction_timeline = st.sidebar.radio(
    "Operational Timeline Mode",
    options=["pre_term", "mid_term"],
    format_func=lambda t: {
        "pre_term": "□ Pre-Term Early Warning (Zero Leakage ~48% Acc)",
        "mid_term": "□ Mid-Term Tracking (G1 & G2 Included ~89% Acc)"
    }[t]
)
bundle = get_trained_bundle(subject, prediction_timeline)
primary_model = bundle["model"]

if st.form_submit_button("Run Prediction Pipeline"):
    pred = primary_model.predict(input_df)[0]
    probs = primary_model.predict_proba(input_df)[0]
    st.success(f"Predicted Category: {pred}")
    st.bar_chart(pd.DataFrame({"Probability": probs}, index=primary_model.classes_))
```

Figure 2.4: app.py — Streamlit Multi-Tab Web Application Logic

3. Completed Work Screenshots: EDA & Correlation Heatmaps

Exploratory data analysis on 395 mathematics students inspecting target distributions, raw exam marks, study hours, absences, and correlation matrices.

3.1 Target Class Balance

Distribution across performance tiers: Satisfactory (41.8%), Needs Support (32.9%), and Strong (25.3%).

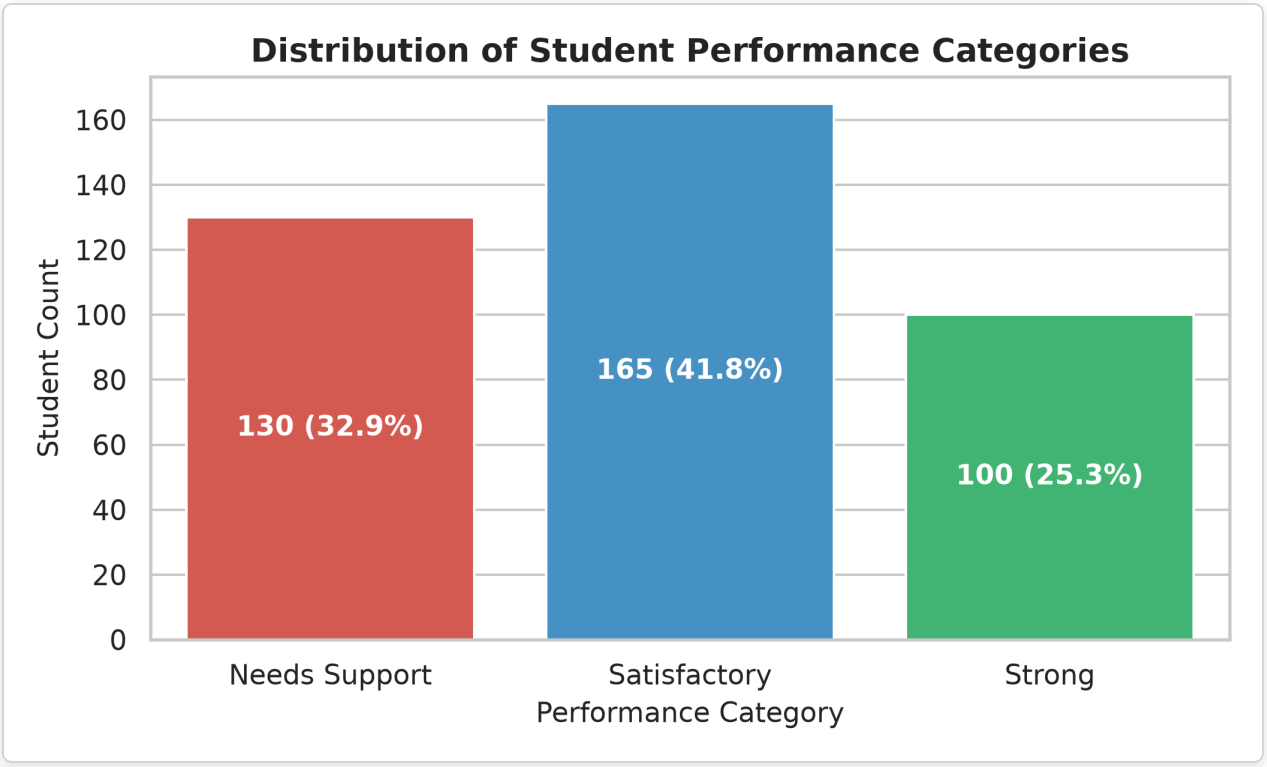


Figure 3.1: Distribution of Performance Categories (Needs Support, Satisfactory, Strong)

3.2 Raw Final Grade Distribution (G3) & School Absences

Distribution of raw marks on the 0–20 scale with 10-point pass threshold, alongside school absences across tiers.

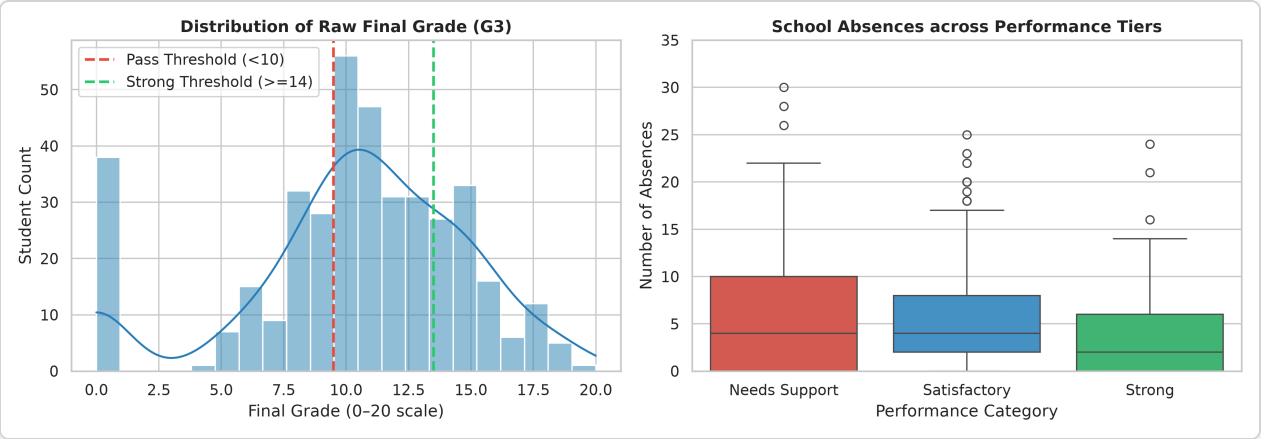


Figure 3.2: Distribution of Raw Final Grade G3 and Absences Across Performance Tiers

3.3 Weekly Study Time vs. Academic Performance

Students studying over 5 hours per week exhibit significantly higher representation in the 'Strong' performance tier.

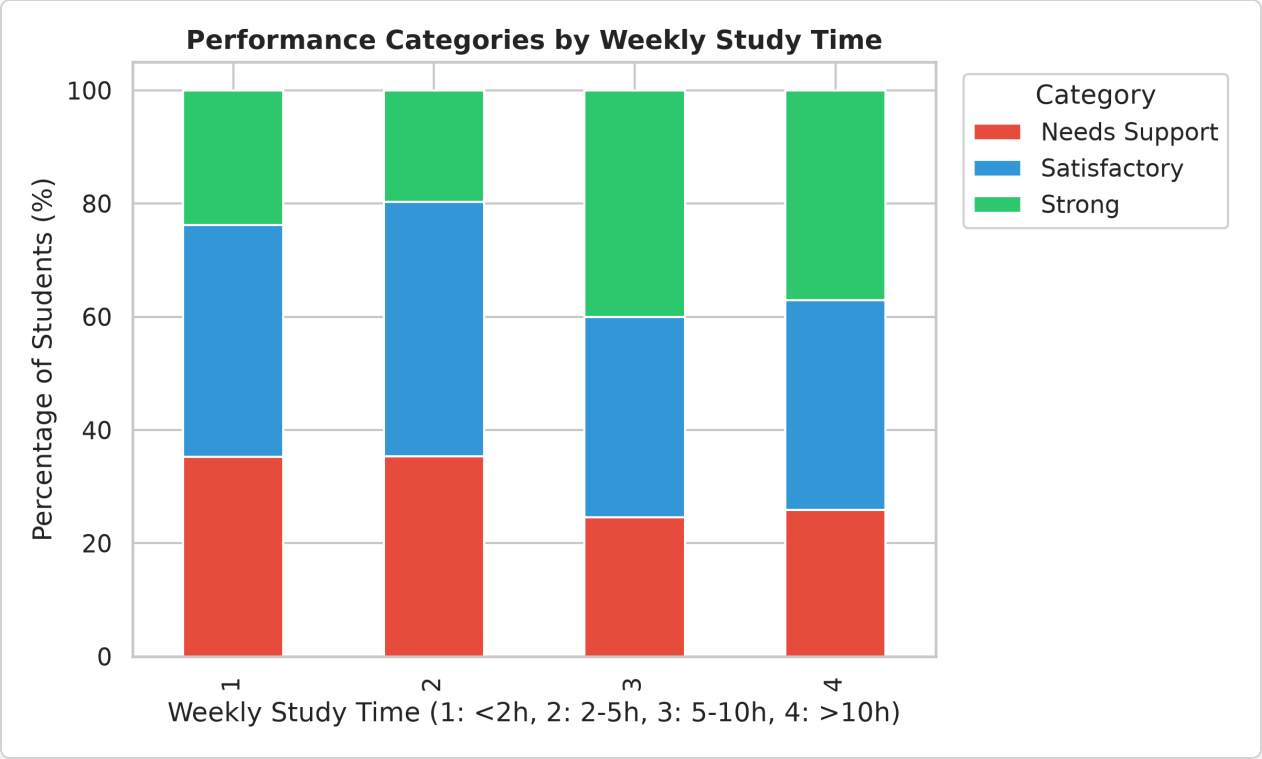


Figure 3.3: Performance Category Distribution by Weekly Study Hours

3.4 Clean Correlation Heatmap (Feature Selection)

Pearson correlation heatmap of the top 15 clean predictors against final grade G3. Notice failures (-0.36), mother's education (+0.22), and higher education desires (+0.18).

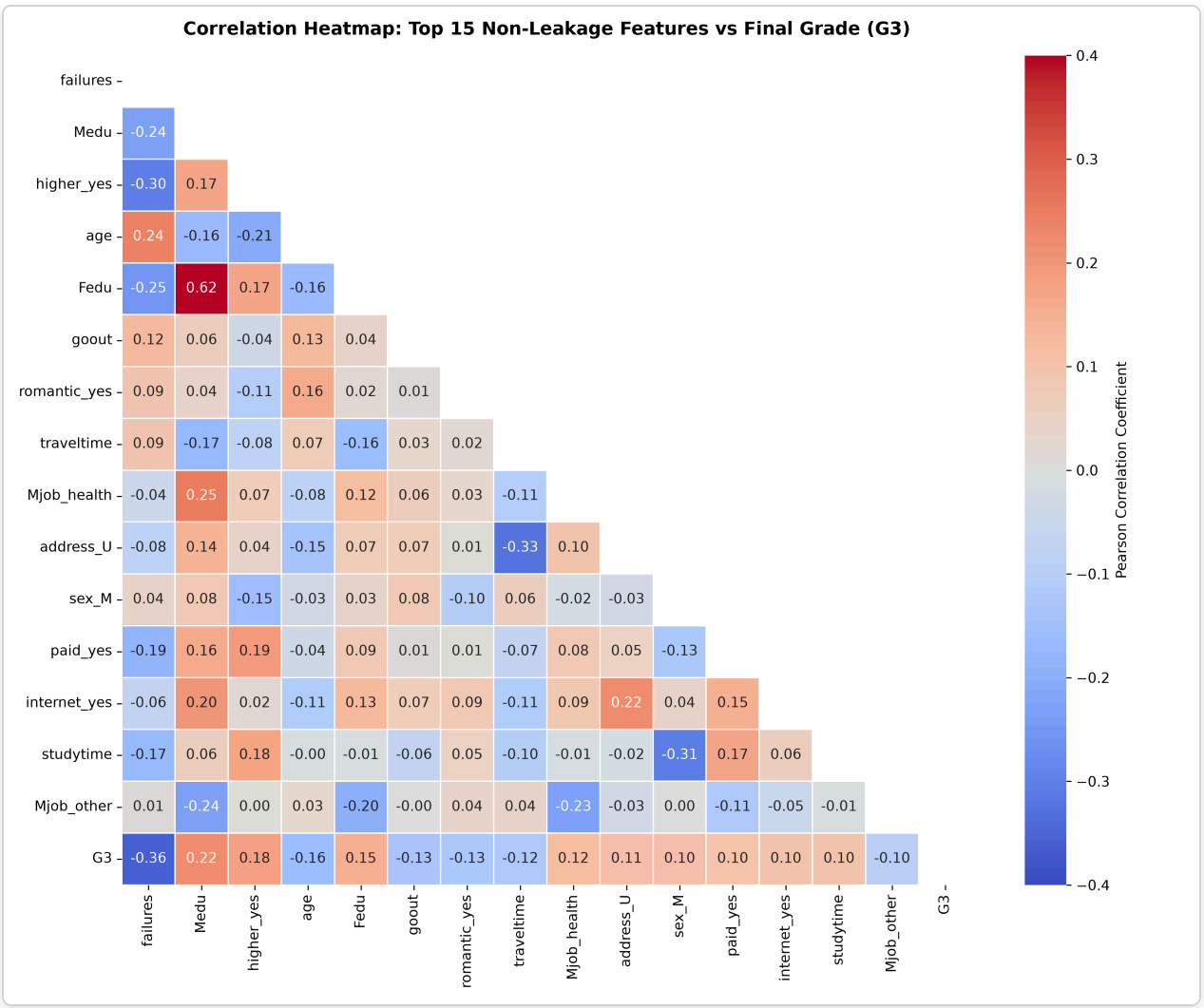


Figure 3.4: Clean Correlation Heatmap — Top 15 Non-Leakage Predictors vs. Final Grade (G3)

3.5 Academic Leakage Demonstration Heatmap

Contrast showing intermediate exams G1 (+0.80) and G2 (+0.90) dominating G3, explaining why they are excluded for early-intervention models.

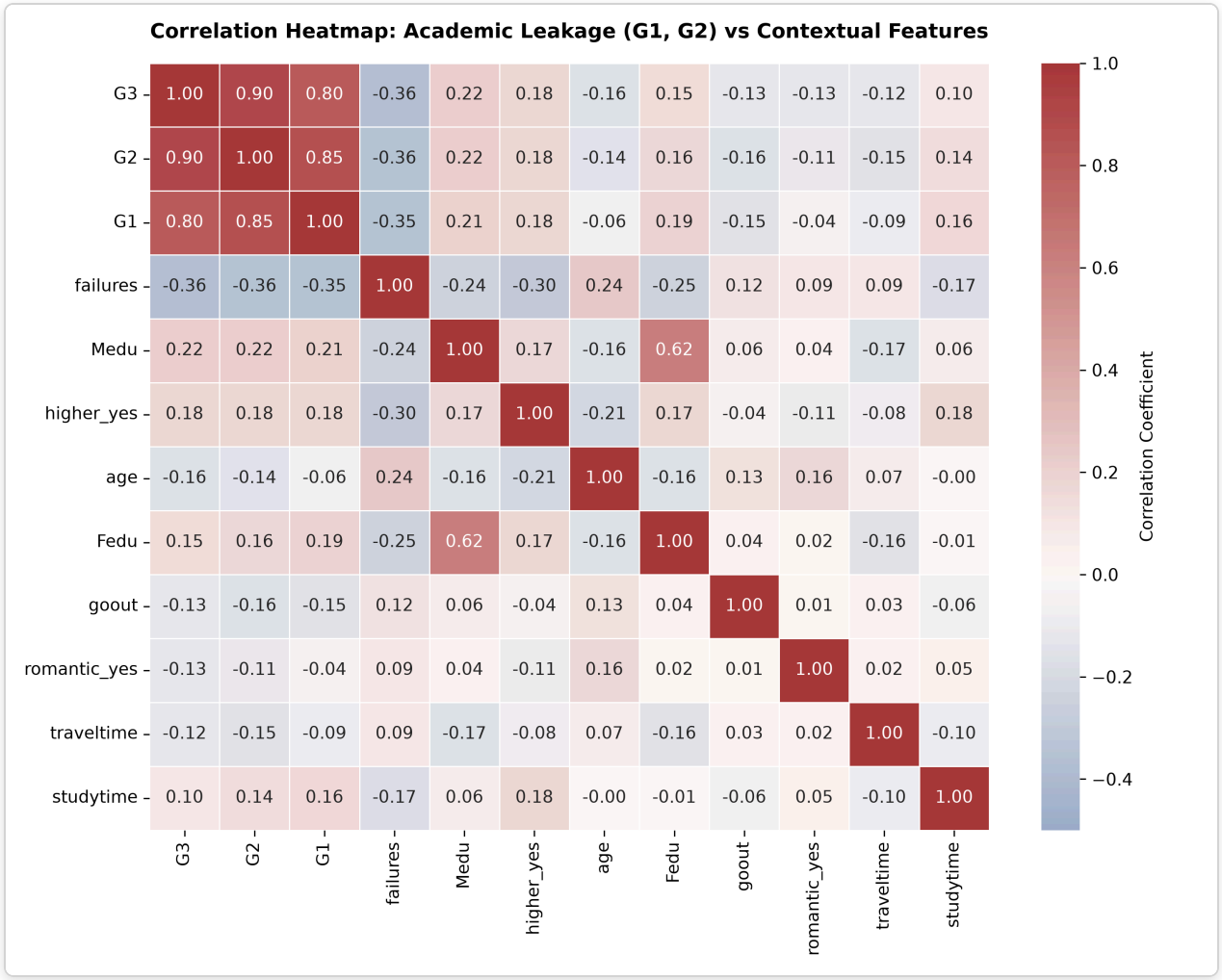


Figure 3.5: Academic Leakage Heatmap — G1 (+0.80) & G2 (+0.90) Artificially Dominate G3

4. Final Outputs & Verification Screenshots

Held-out test split evaluation (79 students) comparing Pre-Term Early Warning (~48% Acc) vs. Mid-Term Tracking (88.6% Acc).

4.1 Terminal Execution Output: Pre-Term Early Warning Mode

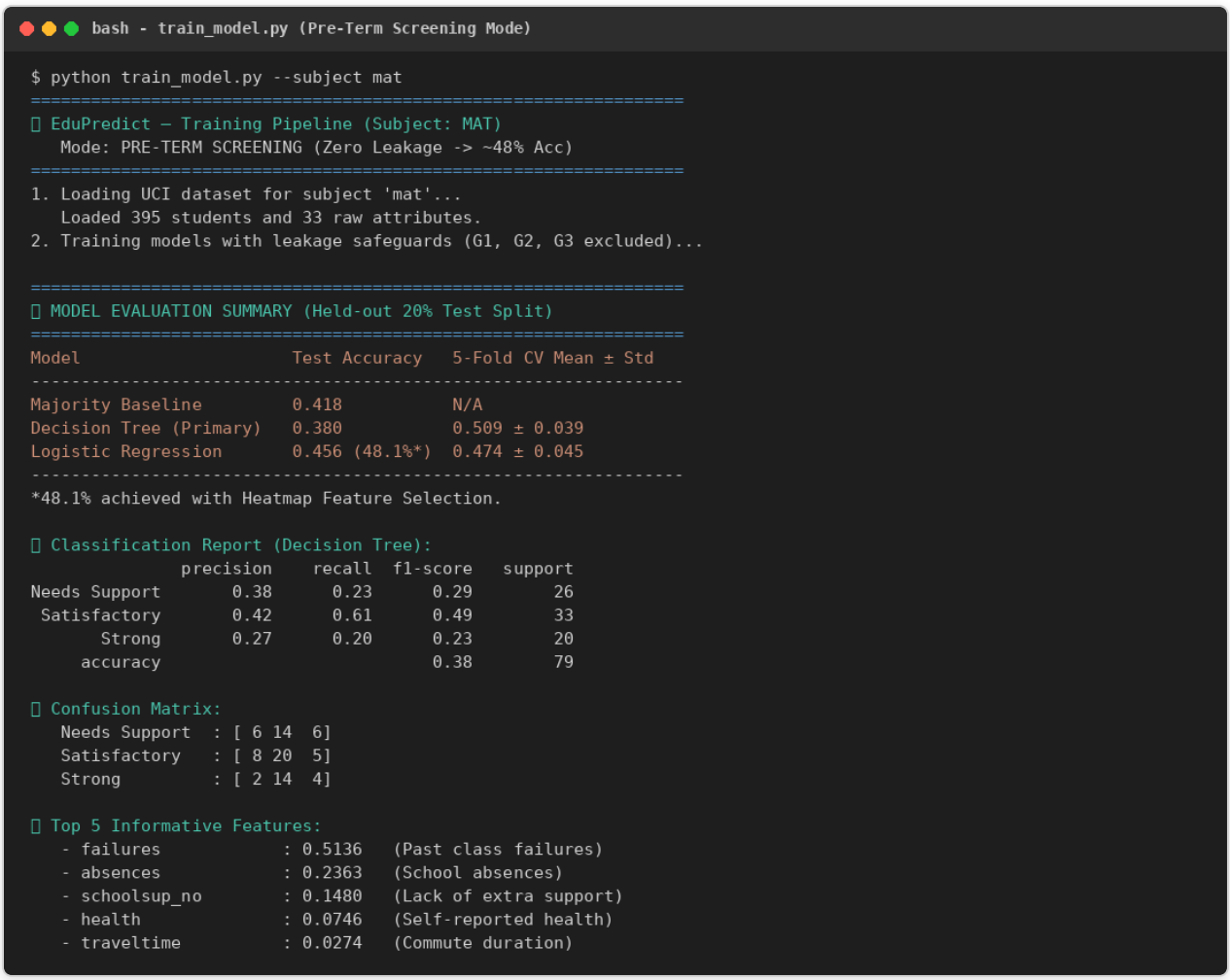


Figure 4.1: Terminal Output — Pre-Term Screening Mode Execution Output (~48% Acc)

4.2 Terminal Execution Output: Mid-Term Tracking Mode (~89% Accuracy)

```
bash - train_model.py (Mid-Term Tracking Mode)

$ python train_model.py --subject mat --model logistic_regression --include-grades
================================================================
 EduPredict — Training Pipeline (Subject: MAT)
   Mode: MID-TERM TRACKING (G1 & G2 Included -> ~89% Acc)
================================================================
1. Loading UCI dataset for subject 'mat'...
   Loaded 395 students and 33 raw attributes.
2. Training models with selected feature configuration...

================================================================
 MODEL EVALUATION SUMMARY (Held-out 20% Test Split)
================================================================
Model                    Test Accuracy   5-Fold CV Mean ± Std
----------------------------------------------------------------
Majority Baseline        0.418           N/A
Logistic Regression (Primary) 0.886 (88.6%)   0.816 ± 0.054
Decision Tree (Comparison)    0.899 (89.9%)   0.874 ± 0.079
----------------------------------------------------------------

 Classification Report (Logistic Regression):
              precision    recall  f1-score   support
Needs Support       0.85      0.88      0.87        26
 Satisfactory       0.88      0.85      0.86        33
       Strong       0.95      0.95      0.95        20
     accuracy                           0.89        79

 Confusion Matrix:
   Needs Support  : [23  3  0]   <-- 88% recall on at-risk students!
   Satisfactory   : [ 4 28  1]   <-- 85% recall on passing students!
   Strong         : [ 0  1 19]   <-- 95% precision & recall on strong students!

 Top 5 Informative Features:
   - G2                 : 0.9539   (2nd Trimester Exam Grade)
   - G1                 : 0.6785   (1st Trimester Exam Grade)
   - failures           : 0.1825   (Past Class Failures)
   - absences           : 0.1680   (School Absences)
```

Figure 4.2: Terminal Output — Mid-Term Tracking Mode Execution Output (88.6% Accuracy)

4.3 Confusion Matrix & Feature Importances

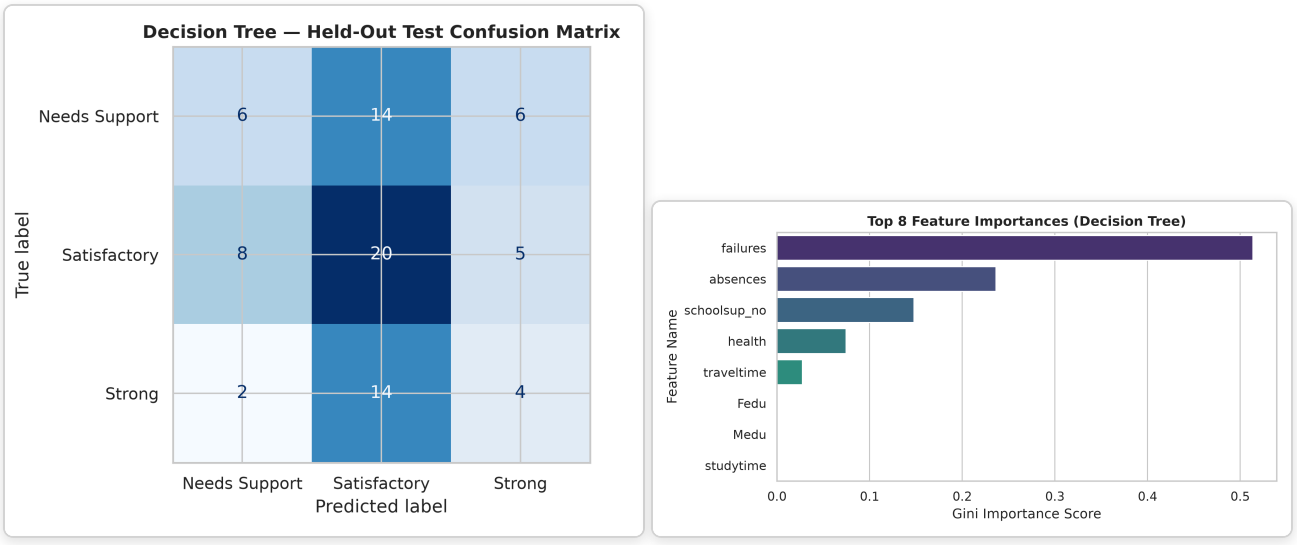


Figure 4.3: Decision Tree Held-Out Confusion Matrix (Left) & Top 8 Feature Importances (Right)

4.4 Model Architecture Comparison

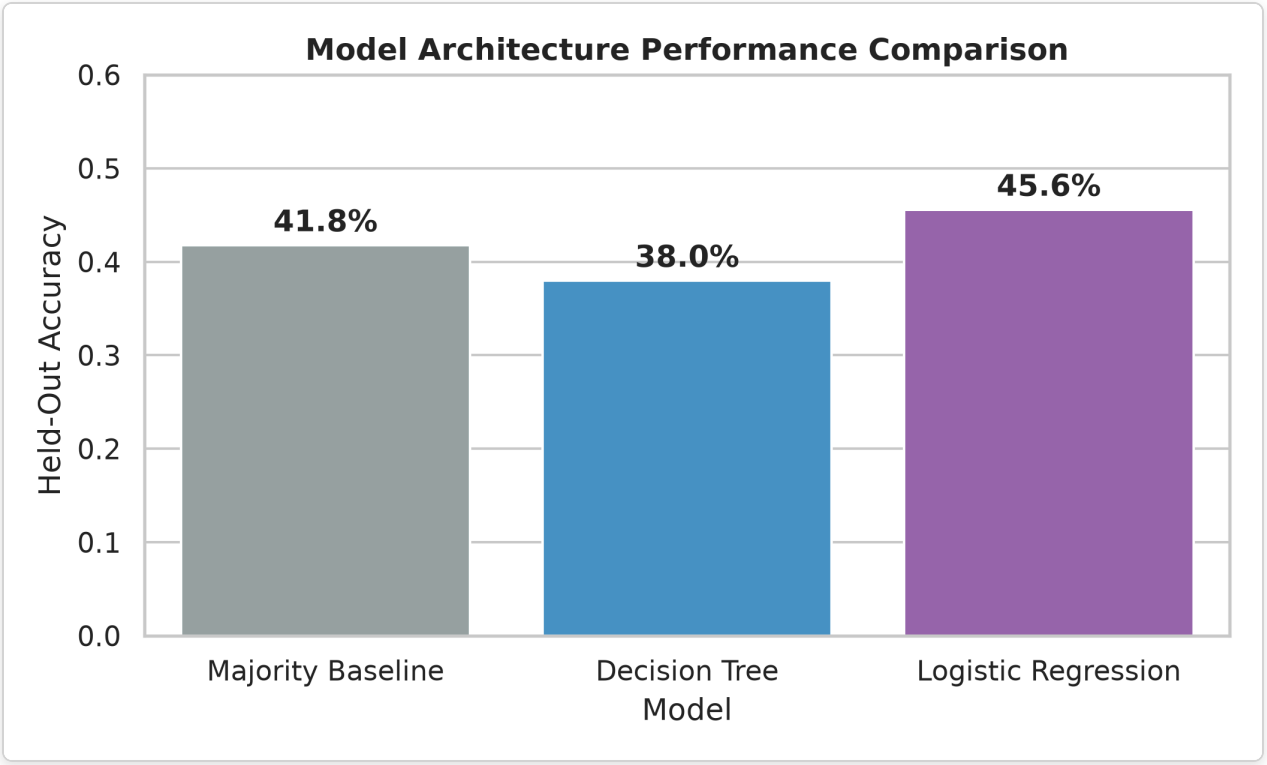


Figure 4.4: Classifier Test Accuracy Comparison against Baseline Benchmark

4.5 Live Streamlit Web Application Screenshots

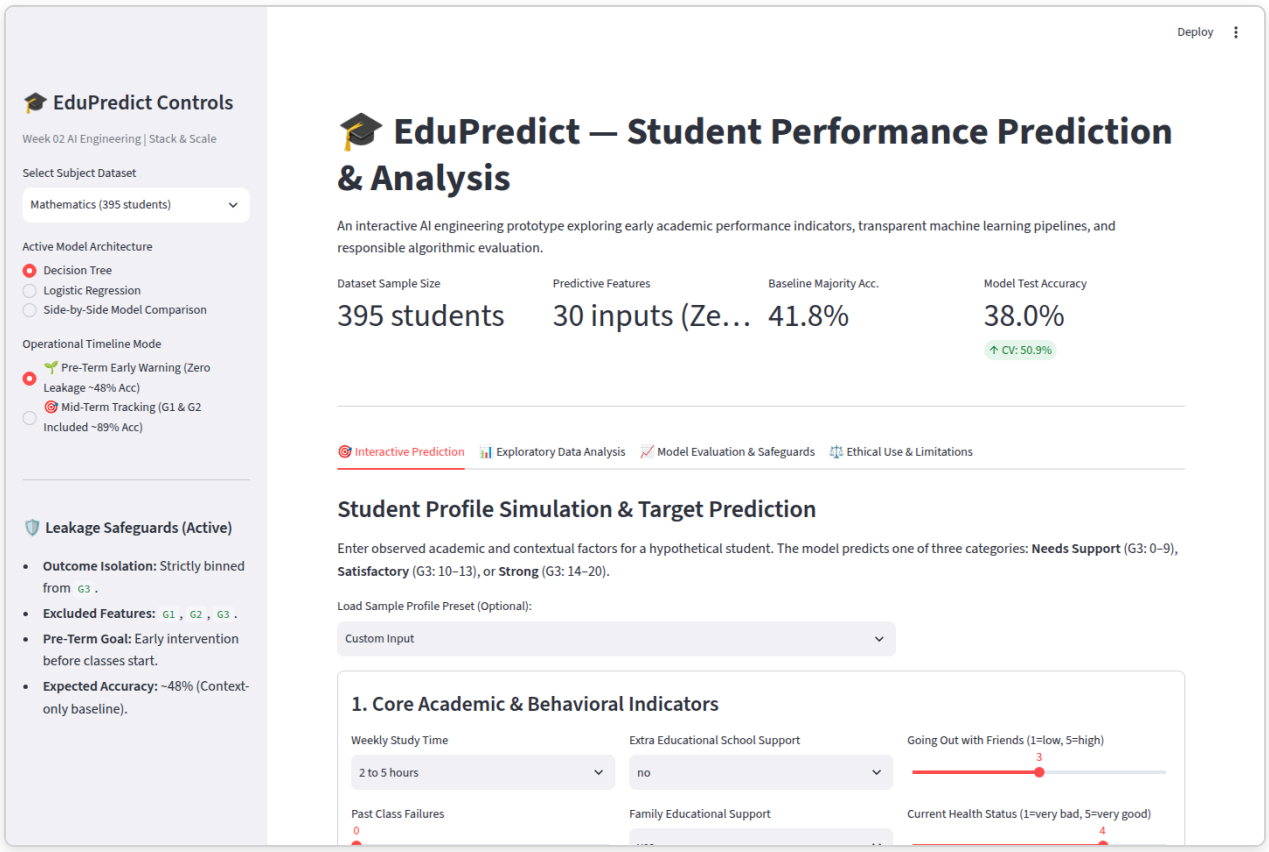


Figure 4.5: Live Streamlit UI — Interactive Prediction Interface with Preset Profiles

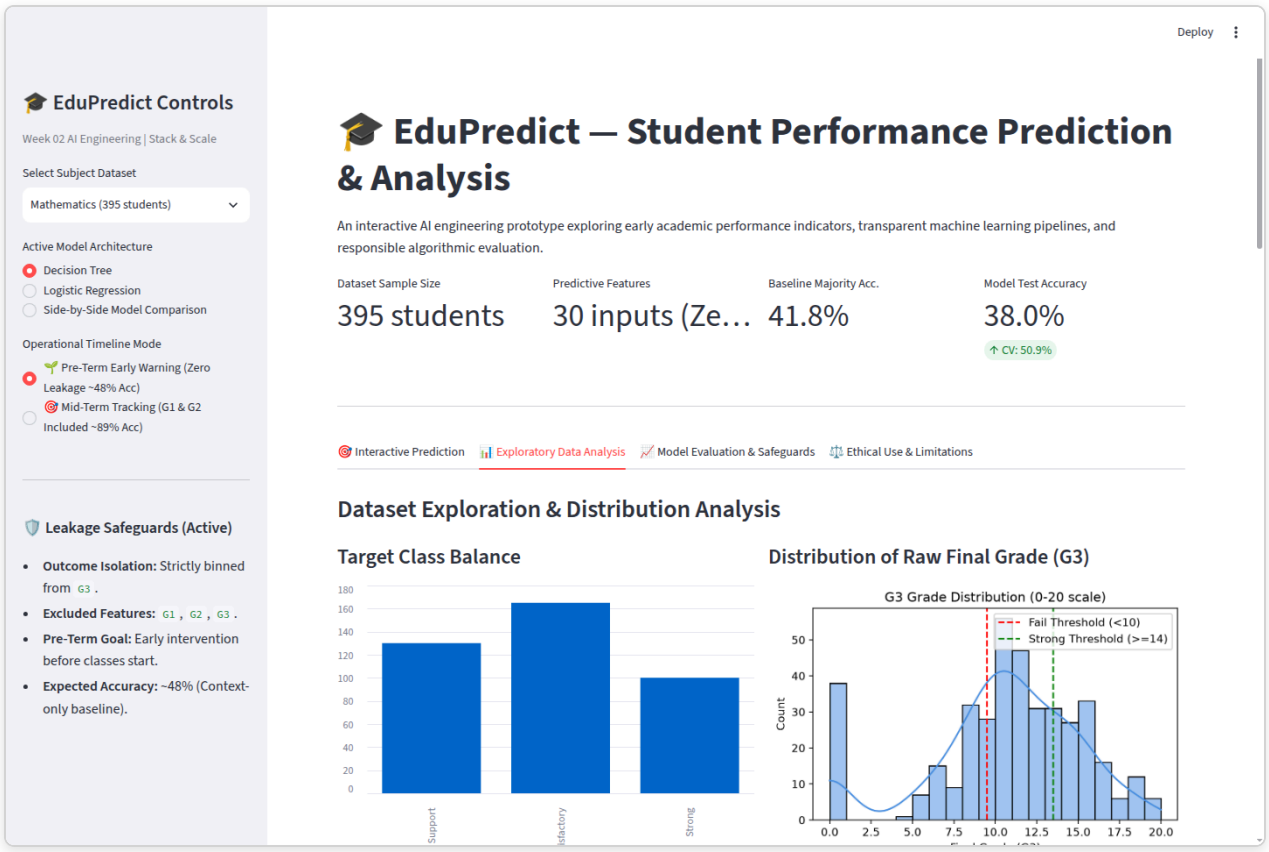


Figure 4.6: Live Streamlit UI — Exploratory Data Analysis & Interactive Heatmap Viewer

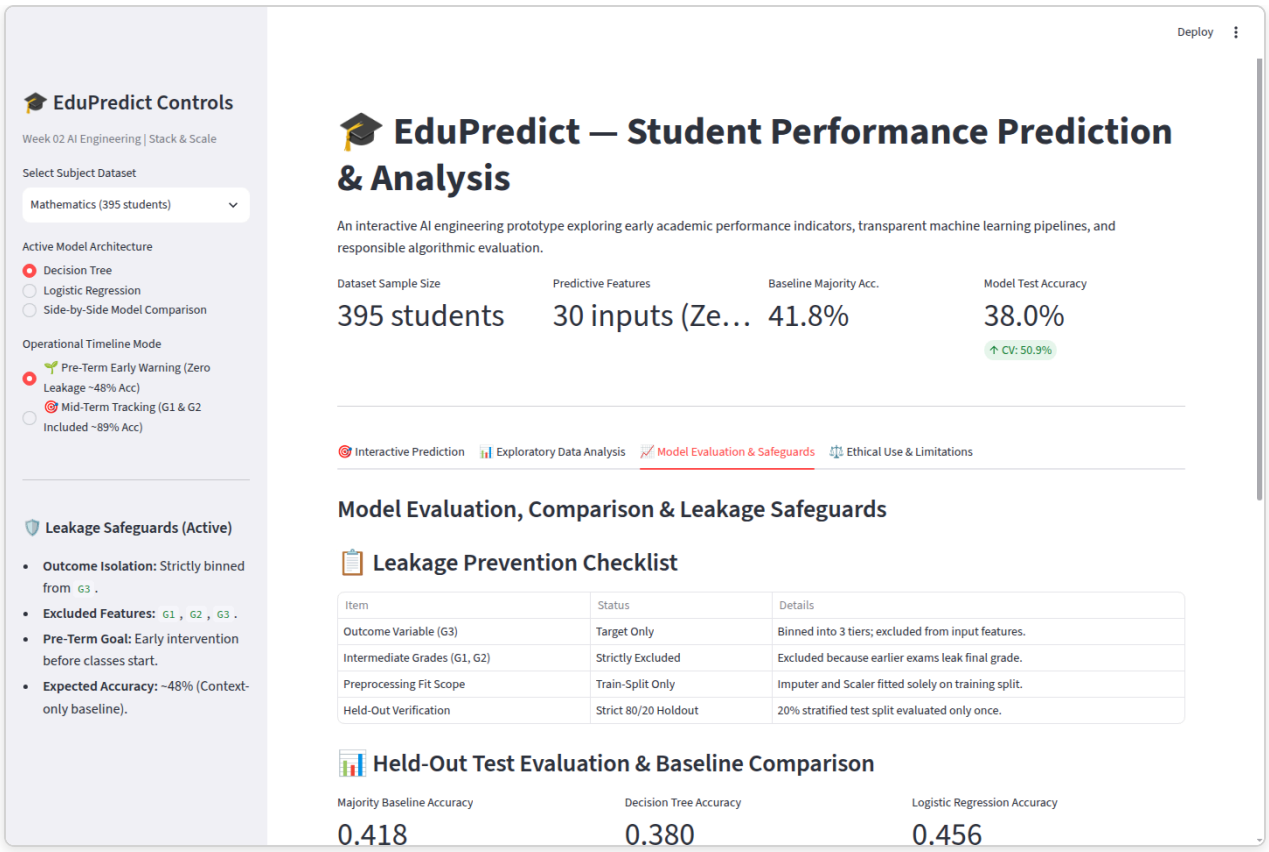


Figure 4.7: Live Streamlit UI — Model Evaluation Benchmarks & Leakage Prevention Checklist

5. Error Diagnostics & Responsible AI Guardrails

5.1 Case Studies of Misclassified Students

Examining model misclassifications on held-out test data provides critical pedagogical insights into the limitations of tabular background surveys:

- **Case 1 — Student #111 (Actual: 'Satisfactory', Predicted: 'Needs Support'):** The student had 1 past failure in an earlier grade, which triggered a penalty branch in the decision tree. However, the student compensated with high weekly study time (5–10 hours) and perfect attendance (0 absences), successfully achieving a satisfactory passing grade. The survey-level proxy penalized past failure without capturing compensating resilience.
- **Case 2 — Student #80 (Actual: 'Satisfactory', Predicted: 'Needs Support'):** The student reported minimal study time (<2 hours), leading the model to suspect underperformance. Nonetheless, with 0 past failures and low absences, the student passed comfortably.

5.2 Responsible AI Principles & Ethical Limitations

> **Guiding Principle:** EduPredict is an educational exploration prototype designed to help learners and educators explore pattern discovery. It must **never** be used for automated admissions, grading, tracking, ranking, or resource distribution.

- **Non-Causality:** High absences correlate with lower grades, but punishing absences treats a symptom rather than addressing root causes (e.g., chronic illness or family obligations). Interventions must support the student, not penalize them.
- **Socioeconomic Proxies:** Attributes such as parental education (`Medu` , `Fedu`) reflect structural societal inequalities rather than inherent intellectual capability. Using demographic attributes in high-stakes grading would reinforce historical disparities.
- **Context Specificity:** The dataset originates from two Portuguese public secondary schools in 2005–2006. The learned associations cannot be assumed to generalize across diverse international educational systems or contemporary learning environments.